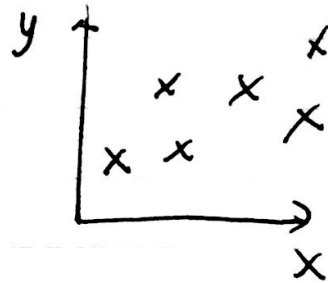


supervised learning

* regression

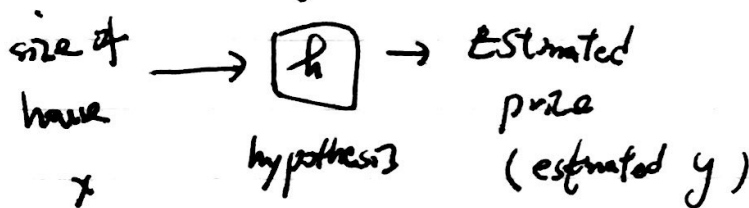
* classification



training set



learning algorithm

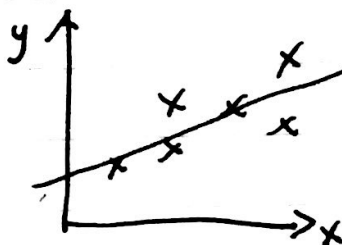


h maps from x 's to y 's.

$$h_{\theta}(x) = \theta_0 + \theta_1 x \quad (\text{shorthand } h(x))$$

univariate linear regression (one variable)

$\{\theta_0, \theta_1\}$: parameters.



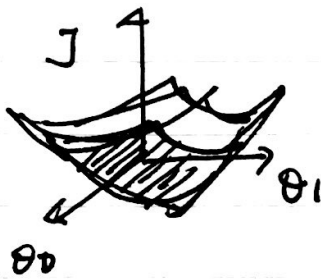
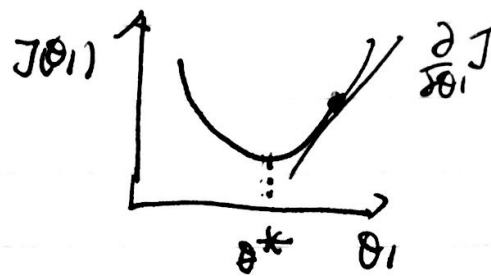
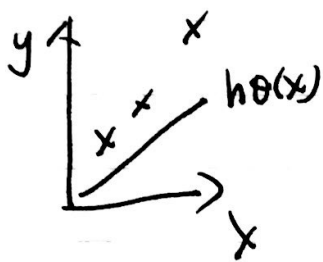
minimize $\sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$

↓

$h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

minimize $J(\theta_0, \theta_1)$
cost fun, squared error fun



Multi-variate, multiple regression.
 linear regression

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4 \dots$$

$$\left(x_0^{(i)} = 1, \text{convention} \right)$$

$$= \theta^T x$$

$$= [\theta_0 \theta_1 \dots \theta_n] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

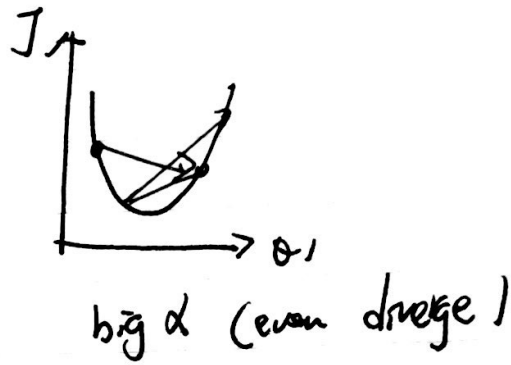
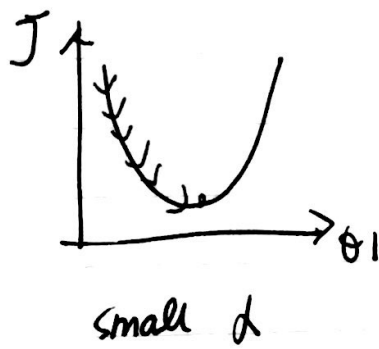
$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

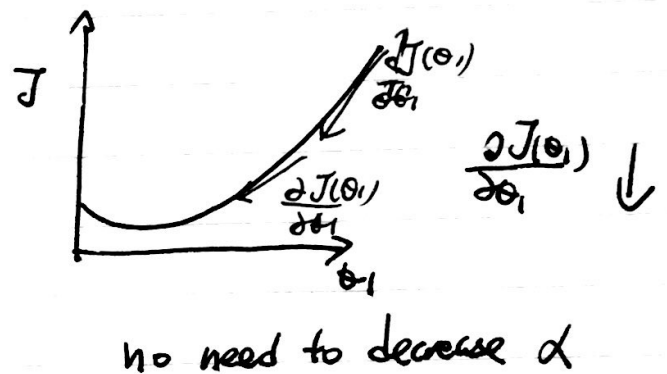
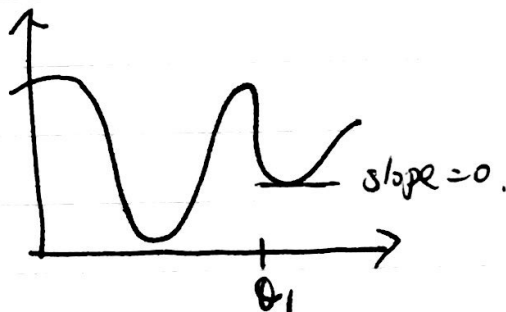
Gradient descent: learning rate (n+1) dimension

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\vec{\theta}) \quad \text{simultaneously update } \vec{\theta}$$

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2, \quad \frac{\partial}{\partial \theta_j} J(\vec{\theta}) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$



* if already at local optima



"Batch" Gradient Descent:

"Batch": Each step of gradient descent uses all the training examples.

$$\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\begin{cases} \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \\ \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)} \\ \theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)} \\ \vdots \end{cases}$$

4/26 - 4

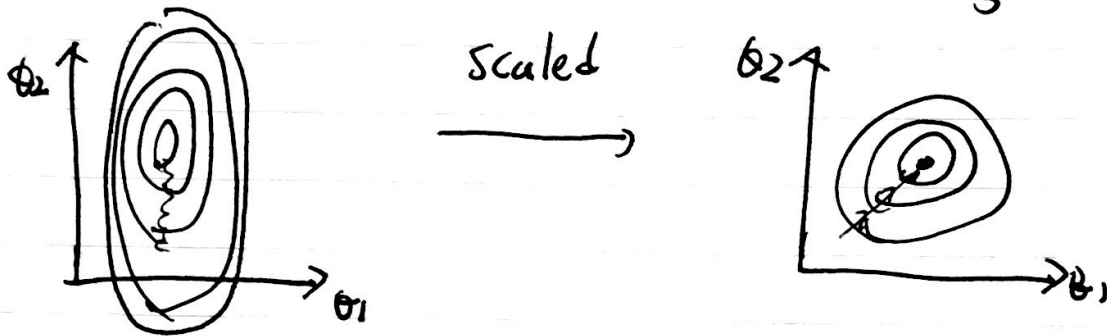
* feature scaling

idea: Make sure features are on a similar scale.

eg. $x_1 \gg x_2$
 $\downarrow$ (# of bed rooms)
 size (2000 feet²)

$$x_1 = \frac{x_1}{2000}$$

$$x_2 = \frac{\# \text{ bed rooms}}{5}$$



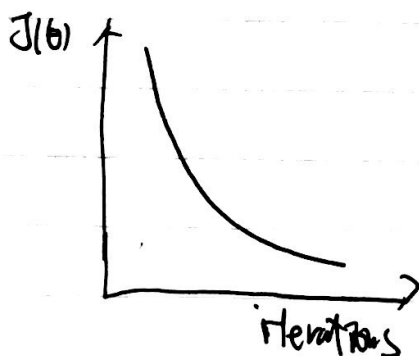
* mean normalization

Replace x_i with $x_i - \mu_i$, $x_1 = \frac{\text{size} - 1000}{2000}$, $x_2 = \frac{\# \text{ of bed} - 2}{5}$..

$$x_1 \leftarrow \frac{x_1 - \mu_1}{s_1}$$

↑
range, or standard dev.

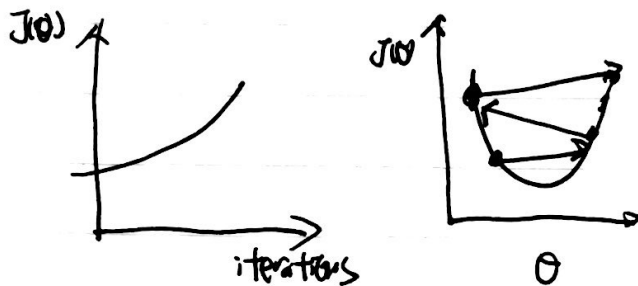
$$x_2 \leftarrow \frac{x_2 - \mu_2}{s_2} \dots$$



10^3 as criterion.

4/26-5.

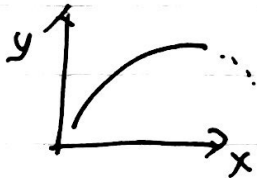
Gradient descent not working $\rightarrow$ using smaller α



* for sufficiently small α , $J(\theta)$ should decrease on every iteration.
 α is too small, convergence is too slow.

Polynomial Regression

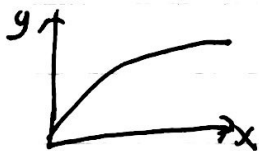
$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$



$$\theta_0 + \theta_1 x + \theta_2 \sqrt{x}$$



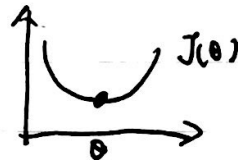
Normal Equation

4/26-6

Normal Eq: Method to solve θ analytically.

* if $D=1, \theta \in \mathbb{R}$

$$\frac{dJ(\theta)}{d\theta} = 0, \text{ solve } \theta$$



* if $D>1, \theta \in \mathbb{R}^{n+1}$

$$J = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\frac{\partial J(\theta)}{\partial \theta_1} = \frac{\partial J(\theta)}{\partial \theta_2} \dots = 0, \text{ solve } \theta_0, \theta_1, \dots, \theta_n$$

$$\vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}, \quad \vec{X}^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix} \rightarrow \hat{X} = \begin{bmatrix} - (x^{(1)})^T - \\ - (x^{(2)})^T - \\ \vdots \\ - (x^{(m)})^T - \end{bmatrix} \left\{ \begin{array}{l} m \\ \text{observations} \end{array} \right.$$

$$\vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

$$\vec{y} = \hat{X} \cdot \vec{\theta}$$

$$\rightarrow \hat{X}^T \vec{y} = \hat{X}^T \hat{X} \vec{\theta}$$

$$= m \times (n+1)$$

$$\hat{X}^T = (n+1) \times m$$

$$\rightarrow \hat{X}^T \hat{X} = (n+1) \times (n+1)$$

$$\vec{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} \left\{ \begin{array}{l} n+1 \end{array} \right.$$

$$\rightarrow \boxed{\vec{\theta} = (\hat{X}^T \hat{X})^{-1} \hat{X}^T \vec{y}}$$

normal Eq.

If Using Normal Eq, no need to feature scaling

m training examples, n features.

Gradient Descent

- * choose α
- * need iteration

$$n \sim 10^6$$

Normal Eq.

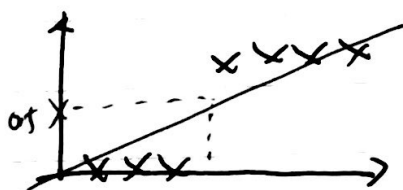
- * no need to choose α
- * no need to iterate
- * compute $(X^T X)^{-1}$ ($n \times n$ or $(n+1) \times (n+1)$)
slow! $O(n^3)$
if consider θ_0
 $n \sim 10000$, still ok

* What if $(X^T X)^{-1}$ is non-invertible?

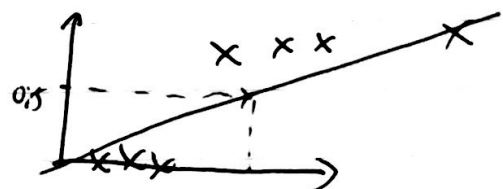
- ① Redundant features (linearly dependent)
 $x_1, x_2, x = (3.28)^2 x_2$, delete x_1 or x_2
- ② Too many features ($m \leq n$)
→ Delete some features, or else regularization.

Logistic Regression $y=0, y=1$ → classification

example why not using linear Regression.



but



$y = 1$ if $\hat{h}(x) > 0.5$, $y = 0$ if $\hat{h}(x) < 0.5$
 $\hat{h}(x)$, also $\hat{h}(x)$ can be > 1 !
 < 0